

Chapter 4 Evaluation Metrics

After you make predictions, you need to know if they are any good. There are standard measures that we can use to summarize how good a set of predictions actually is. Knowing how good a set of predictions is allows you to make estimates about the skill of a given machine learning model of your problem. In this tutorial, you will discover how to implement four standard prediction evaluation metrics from scratch in Perl. After reading this tutorial, you will know:

- How to implement classification accuracy.
- How to implement and interpret a confusion matrix.
- How to implement mean absolute error for regression.
- How to implement root mean squared error for regression.

Let's get started.

4.1 Description

You must estimate the quality of a set of predictions when training a machine learning model. Performance metrics like classification accuracy and root mean squared error can give you a clear objective idea of how good a set of predictions is, and in turn how good the model is that generated them. This is important as it allows you to tell the difference and select among:

- Different transforms of the data used to train the same machine learning model.
- Different machine learning models trained on the same data.
- Different configurations for a machine learning model trained on the same data.

As such, performance metrics are a required building block in implementing machine learning algorithms from scratch.

4.2. Tutorial

This tutorial is divided into 4 parts:

1. Classification Accuracy.
2. Confusion Matrix.
3. Mean Absolute Error.
4. Root Mean Squared Error.

These steps will provide the foundations you need to handle evaluating predictions made by machine learning algorithms.

4.2.1 Classification Accuracy

A quick way to evaluate a set of predictions on a classification problem is by using accuracy. Classification accuracy is a ratio of the number of correct predictions out of all predictions that were made. It is often presented as a percentage between 0% for the worst possible accuracy and 100% for the best possible accuracy.

accuracy = correct predictions x 100 / total predictions

(4.1)

We can implement this in a function that takes the expected outcomes and the predictions as arguments. Below is this function named `accuracy_metric()` that returns classification accuracy as a percentage. Notice that we use `==` to compare the equality actual to predicted values. This allows us to compare integers or strings, two main data types that we may choose to use when loading classification data.

```
In [1]: use strict;
use warnings;
use Data::Dump qw(dump);
use List::Util qw(zip min max sum uniq);
use sml; # Statistical Machine Learning Library
use Chart::Plotly qw(show_plot); # show_plot sirve para Visual Code o ejecución local
use Chart::Plotly::Plot;
use Chart::Plotly::Trace::Scatter;
IPerl->load_plugin('Chart::Plotly'); # Solo sirve para Jupyter Notebooks
```

```
In [2]: # Defined in Section 4.2.1 Classification Accuracy
# Function To Calculate Classification Accuracy.
# Calculate accuracy percentage between two lists
sub accuracy_metric{
    my ($self, $actual, $predicted) = @_;
    my $correct = 0;
    for my $pair (zip $actual, $predicted){
        $correct++ if ($pair->[0] == $pair->[1]);
    }
    return sprintf '%0.2f', $correct / @$actual * 100.0;
}

sml->add_to_class('accuracy_metric', \%{'accuracy_metric'});
```

```
Out[2]: *sml::accuracy_metric
```

We can contrive a small dataset to test this function. Below are a set of 10 actual and predicted integer values. There are two mistakes in the set of predictions.

Running this example produces the expected accuracy of 80% or 8/10.

```
In [3]: # Example of calculating classification accuracy

# Test accuracy
my $actual      = [0,0,0,0,0,1,1,1,1,1];
my $predicted   = [0,1,0,0,0,1,0,1,1,1];
print "actual\tpredicted\n";
print $_->[0], "\t", $_->[1], "\n" for zip ($actual, $predicted);
my $accuracy    = sml->accuracy_metric($actual, $predicted);
printf 'Accuracy: %0.2f%%', $accuracy;
# Example Output From Calculating Classification Accuracy.
# actual      predicted
# 0           0
# 0           1
# 0           0
# 0           0
# 0           0
# 1           1
# 1           0
# 1           1
# 1           1
```

```
# 1      1
# 80.0
```

```
actual predicted
0      0
0      1
0      0
0      0
0      0
1      1
1      0
1      1
1      1
1      1
Accuracy: 80.00%
```

```
Out[3]: 1
```

Accuracy is a good metric to use when you have a small number of class values, such as 2, also called a binary classification problem. Accuracy starts to lose its meaning when you have more class values and you may need to review a different perspective on the results, such as a confusion matrix.

4.2.2 Confusion Matrix

A confusion matrix provides a summary of all of the predictions made compared to the expected actual values. The results are presented in a matrix with counts in each cell. The counts of predicted class values are summarized horizontally (rows), whereas the counts of actual values for each class values are presented vertically (columns). A perfect set of predictions is shown as a diagonal line from the top left to the bottom right of the matrix. The value of a confusion matrix for classification problems is that you can clearly see which predictions were wrong and the type of mistake that was made. Let's create a function to calculate a confusion matrix. We can start off by defining the function to calculate the confusion matrix given a list of actual class values and a list of predictions. The function is listed below and is named `confusion_matrix()`. It first makes a list of all of the unique class values and assigns each class value a unique integer or index into the confusion matrix. The confusion matrix is always square, with the number of class values indicating the number of rows and columns required. Here, the first index into the matrix is the row for actual values and the second is the column for predicted values. After the square confusion matrix is created and initialized to zero counts in each cell, it is a matter of looping through all predictions and incrementing the count in each cell. The function returns two objects. The first is the set of unique class values, so that they can be displayed when the confusion matrix is drawn. The second is the confusion matrix itself with the counts in each cell.

```
In [4]: # Example of Calculating and Displaying a Pretty Confusion Matrix

# Function To Calculate a Confusion Matrix.
# calculate a confusion matrix
sub confusion_matrix{
  my ($self, $actual, $predicted) = @_;

  # 1. Get sorted unique labels to ensure consistent matrix indexing
  my @unique = sort { $a cmp $b } uniq(@$actual, @$predicted);

  # 2. Map labels directly to their matrix index positions
  my $i = 0;
  my %lookup = map { $_ => $i++ } @unique;

  # 3. Initialize a clean matrix full of zeros
```

```

my $matrix = [ map { [ (0) x @unique ] } 0 .. $#unique ];

# 4. Populate the matrix directly
for my $idx (0 .. $$actual) {
    my $x = $lookup{$actual->[$idx]};
    my $y = $lookup{$predicted->[$idx]};

    # Guard against undef if predicted contains a label not present in actual
    $matrix->[$x][$y]++ if defined $x and defined $y;
}

return wantarray ? (\@unique, $matrix) : $matrix;
}

sml->add_to_class('confusion_matrix', \&{'confusion_matrix'});

```

Out[4]: *sml::confusion_matrix

Let's make this concrete with an example. Below is another contrived dataset, this time with 3 mistakes.

```

In [5]: # Example of a Set of Contrived Predictions and Expected Values.
my $actual      = [0,0,0,0,0,1,1,1,1,1];
my $predicted    = [0,1,1,0,0,1,0,1,1,1];
print "actual\tpredicted\n";
print $_->[0], "\t", $_->[1], "\n" for zip ($actual, $predicted);

```

actual	predicted
0	0
0	1
0	1
0	0
0	0
1	1
1	0
1	1
1	1
1	1

We can calculate and print the confusion matrix for this dataset as follows:

```

In [6]: # Test confusion matrix with integers
my ($unique, $matrix) = sml->confusion_matrix($actual, $predicted);
printf "classes: %s\n", dump $unique;
printf "confusion matrix: %s\n", dump $matrix;
# Example Output From Calculating a Confusion Matrix.
# [0, 1]
# [[3, 2], [1, 4]]

```

```

classes: [0, 1]
confusion matrix: [[3, 2], [1, 4]]

```

Out[6]: 1

It's hard to interpret the results this way. It would help if we could display the matrix as intended with rows and columns. Below is a function to correctly display the matrix. The function is named `print_confusion_matrix()`. It names the columns as Z for Actual and the rows as P for Predicted. Each column and row are named for the class value to which it corresponds. The matrix is laid out with the expectation that each class label is a single character or single digit integer and that the counts are also single digit integers. You could extend it to handle large class labels or prediction counts as an exercise.

```
In [7]: # Function To Pretty Print a Confusion Matrix.
# pretty print a confusion matrix
sub print_confusion_matrix{
  my ($self, $unique, $matrix) = @_;
  print 'A/P ' . join(' ', map {$_} @$unique), "\n";
  while (my ($i, $x) = each @$unique){
    print sprintf " %s| %s\n", $x, join(' ', map {$_} @{$matrix->[$i]});
  }
  print "\n";
}

sml->add_to_class('print_confusion_matrix', \&{'print_confusion_matrix'});
```

```
Out[7]: *sml::print_confusion_matrix
```

Running the example produces the output below. We can see the class labels of 0 and 1 across the top and bottom. Looking down the diagonal of the matrix from the top left to bottom right, we can see that 3 predictions of 0 were correct and 4 predictions of 1 were correct. Looking in the other cells, we can see 2 + 1 or 3 prediction errors. We can see that 2 predictions were made as a 1 that were in fact actually a 0 class value. And we can see 1 prediction that was a 0 that was in fact actually a 1.

```
In [8]: # Example Output From Printing a Pretty Confusion Matrix.
# Test confusion matrix with integers
my $actual      = [0,0,0,0,0,1,1,1,1,1];
my $predicted    = [0,1,1,0,0,1,0,1,1,1];
my ($unique, $matrix) = sml->confusion_matrix($actual, $predicted);
sml->print_confusion_matrix($unique, $matrix);
# (A)0 1 # Example Output From Printing a Pretty Confusion Matrix.
# 0| 3 2
# 1| 1 4
```

```
A/P 0 1
0| 3 2
1| 1 4
```

```
Out[8]: 1
```

A confusion matrix is always a good idea to use in addition to classification accuracy to help interpret the predictions.

4.2.3 Mean Absolute Error

Regression problems are those where a real value is predicted. An easy metric to consider is the error in the predicted values as compared to the expected values. The Mean Absolute Error or MAE for short is a good first error metric to use. It is calculated as the average of the absolute error values, where absolute means made positive so that they can be added together.

$$MAE = \sum_{i=1}^n abs(predicted_i - actual_i) / totalpredictions$$

(4.2)

Below is a function named mae metric() that implements this metric. As above, it expects a list of actual outcome values and a list of predictions. We use the built-in abs() Python function to calculate the absolute error values that are summed together.

```
In [9]: # Function To Calculate Mean Absolute Error.
# Calculate mean absolute error
```

```

sub mae_metric{
  my ($self, $actual, $predicted) = @_;
  my $sum_error = 0.0;
  for my $pair (zip $actual, $predicted){
    $sum_error += abs($pair->[0] - $pair->[1]);
  }
  return sprintf '%0.2f', $sum_error / @$actual;
}

sml->add_to_class('mae_metric', \&'mae_metric');

```

Out[9]: *sml::mae_metric

We can contrive a small regression dataset to test this function.

```

In [10]: # Small Set of Contrived Regression Predictions and Actual Values.
$actual   = [0.1, 0.2, 0.3, 0.4, 0.5];
$predicted = [0.11, 0.19, 0.29, 0.41, 0.5];
print "actual\tpredicted\n";
print $_->[0], "\t", $_->[1], "\n" for zip ($actual, $predicted);

```

actual	predicted
0.1	0.11
0.2	0.19
0.3	0.29
0.4	0.41
0.5	0.5

Only one prediction (0.5) is correct, whereas all other predictions are wrong by 0.01. Therefore, we would expect the mean absolute error (or the average positive error) for these predictions to be a little less than 0.01. Below is an example that tests the `mae_metric()` function with the contrived dataset.

Running this example prints the output below. We can see that as expected, the MAE was 0.008, a small value slightly lower than 0.01.

```

In [11]: # Test MAE
my $mae = sml->mae_metric($actual, $predicted);
print $mae;
# Example Output From Calculating the Mean Absolute Error.
# 0.01

```

0.01

Out[11]: 1

4.2.4 Root Mean Squared Error

Another popular way to calculate the error in a set of regression predictions is to use the Root Mean Squared Error. Shortened as RMSE, the metric is sometimes called Mean Squared Error or MSE, dropping the Root part from the calculation and the name. RMSE is calculated as the square root of the mean of the squared differences between actual outcomes and predictions. Squaring each error forces the values to be positive, and the square root of the mean squared error returns the error metric back to the original units for comparison.

$$RMSE = \sqrt{(\sum_{i=1}^n (predicted_i - actual_i)^2 / totalpredictions)} \quad (4.3)$$

Below is an implementation of this in a function named `rmse_metric()`. It uses the `sqrt()` function from the `math` module and uses the `**` operator to raise the error to the 2nd power.

```
In [12]: # Defined in Section 4.2.4 Root Mean Squared Error
# Function To Calculate Root Mean Squared Error.
# Calculate root mean squared error
sub rmse_metric{
  my ($self, $actual, $predicted) = @_;
  my $sum_error = 0.0;
  for my $pair (zip $actual, $predicted){
    $sum_error += (($pair->[0] - $pair->[1]) ** 2);
  }
  my $mean_error = $sum_error / @$actual;
  return sprintf '%0.3f', sqrt($mean_error);
}

sml->add_to_class('rmse_metric', \&{'rmse_metric'});
```

```
Out[12]: *sml::rmse_metric
```

We can test this metric on the same dataset used to test the calculation of Mean Absolute Error above. Below is a complete example. Again, we would expect an error value to be generally close to 0.01.

```
In [13]: # Test RMSE
$actual   = [0.1, 0.2, 0.3, 0.4, 0.5];
$predicted = [0.11, 0.19, 0.29, 0.41, 0.5];
my $rmse = sml->rmse_metric($actual, $predicted);
print $rmse;
# Example Output From Calculating the Root Mean Squared Error.
# 0.0089
```

```
0.009
```

```
Out[13]: 1
```

4.2.5 ROC curves

Computing AUC ROC from scratch in Perl without using any libraries.

```
In [14]: # Function to calculate the ROC metrics
# Default behavior: 1 is positive. Setting to 0 reverses the logic.
sub perf_metrics {
  my ($self, $actual, $predicted_prob, $threshold, %args) = (splice(@_, 0, 4), positiv

  # Define operator and truth matching dynamically
  my $op = $args{positive_class} == 1 ? sub { $_[0] >= $_[1] } : sub { $_[0] <= $_[1]

  # Map extraction coordinates dynamically based on the target positive class
  my ($p_idx, $n_idx) = $args{positive_class} == 1 ? (1, 0) : (0, 1);

  # Dynamic operator thresholding via functional lookup
  my $predicted = [map {$_ >= $threshold ? 1 : 0} @$predicted_prob];

  # Compute confusion matrix
  my $matrix = sml->confusion_matrix($actual, $predicted);

  # Extract counts from the confusion matrix
  my $tp = $matrix->[$p_idx][$p_idx]; # True Positives
  my $tn = $matrix->[$n_idx][$n_idx]; # True Negatives
  my $fp = $matrix->[$n_idx][$p_idx]; # False Positives
```

```

my $fn = $matrix->[$p_idx][$n_idx]; # False Negatives

# Compute TPR and FPR
my $tpr = ($tp + $fn) > 0 ? ($tp / ($tp + $fn)) : 0; # Sensitivity: True Positive Ra
my $fpr = ($fp + $tn) > 0 ? ($fp / ($fp + $tn)) : 0; # Specificity: False Positive R

return sprintf('%0.2f', $fpr), sprintf('%0.2f', $tpr), $matrix;
}

sml->add_to_class('perf_metrics', \&{'perf_metrics'});

```

Out[14]: *sml::perf_metrics

```

In [15]: my ($dataset, $header) = sml->load_csv('../data/model.csv');

my ($actual, $predicted_prob) = (zip map {[$_->[1], $_->[2]]} @$dataset);

printf "class: %s\n", dump @$actual[0 .. 4];
printf "predicted_prob: %s\n", dump @$predicted_prob[0 .. 4];
printf "predicted: %s\n", dump map {$_ >= 0.5 ? 1 : 0} @$predicted_prob[0 .. 4];

class: (0, 1, 0, 1, 0)
predicted_prob: (0.592837, 0.624829, 0.073848, 0.544891, 0.015118)
predicted: (1, 1, 0, 1, 0)

```

Out[15]: 1

```

In [16]: # Calculate TPR and FPR for a specific threshold
my ($fpr, $tpr, $matrix) = sml->perf_metrics($actual, $predicted_prob, 0.5, positive_c

# Print sensitivity and specificity
printf "tpr: %.2f, 1 - fpr: %.2f\n", $tpr, 1 - $fpr;
printf "confusion matrix: %s\n", dump $matrix;
# tpr: 0.77, 1 - fpr: 0.69 where positive_class => 1
# tpr: 0.69, 1 - fpr: 0.77 where positive_class => 0

tpr: 0.77, 1 - fpr: 0.69
confusion matrix: [[61, 27], [14, 48]]

```

Out[16]: 1

```

In [17]: # Calculate TPR and FPR for various decision thresholds
my @thresholds = map {$_ / 100} (0 .. 100);
print "@thresholds";

0 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.1 0.11 0.12 0.13 0.14 0.15 0.16 0.17
0.18 0.19 0.2 0.21 0.22 0.23 0.24 0.25 0.26 0.27 0.28 0.29 0.3 0.31 0.32 0.33 0.34 0.35
0.36 0.37 0.38 0.39 0.4 0.41 0.42 0.43 0.44 0.45 0.46 0.47 0.48 0.49 0.5 0.51 0.52 0.53
0.54 0.55 0.56 0.57 0.58 0.59 0.6 0.61 0.62 0.63 0.64 0.65 0.66 0.67 0.68 0.69 0.7 0.71
0.72 0.73 0.74 0.75 0.76 0.77 0.78 0.79 0.8 0.81 0.82 0.83 0.84 0.85 0.86 0.87 0.88 0.8
9 0.9 0.91 0.92 0.93 0.94 0.95 0.96 0.97 0.98 0.99 1

```

Out[17]: 1

```

In [18]: my ($fprs, $tprs) = (zip map {[sml->perf_metrics($actual, $predicted_prob, $_)]} @thre

printf "fprs:%s\n", "@$fprs[0 .. 10]";
printf "tprs:%s\n", "@$tprs[0 .. 10]";

fprs:1.00 1.00 0.95 0.91 0.89 0.88 0.88 0.88 0.86 0.86 0.86
tprs:1.00 1.00 1.00 1.00 1.00 1.00 1.00 1.00 1.00 1.00 1.00

```

Out[18]: 1


```
In [19]: # Function to calculate the integral using the trapezoid rule
sub trapz{
  my ($self, $x, $y) = @_;
  my $sum = 0;
  for my $i (0 .. @$x - 2){
    $sum += ($x->[$i + 1] - $x->[$i]) * ($y->[$i + 1] + $y->[$i]) / 2;
  }
  return sprintf '%0.2f', $sum;
}

sml->add_to_class('trapz', \&{'trapz'});
```

```
Out[19]: *sml::trapz
```

Why Sorting is Required to Obtain the AUC?

The AUC (Area Under the Curve) is calculated using the Trapezoidal Rule. This mathematical method breaks down the area under the ROC plot into a series of vertical trapezoids sliced from left to right along the X-axis (FPR) and sums up their individual areas. The formula to calculate the area of a single trapezoid between two adjacent points, i and $i + 1$, is:

$$\text{Area}_i = (FPR_{i+1} - FPR_i) \times \frac{TPR_{i+1} + TPR_i}{2}$$

Without sorting: The plotting library will connect the points based on their threshold order. Because thresholds yield chaotic jumping FPR paths, your rendering canvas will turn into a messy, zig-zagging "spaghetti" web of overlapping lines crossing back and forth over the screen.

With sorting: The line starts cleanly at the bottom-left corner $(0, 0)$, glides smoothly through the intermediate coordinates from left to right, and finishes at the top-right corner $(1, 1)$, giving you the classic, readable ROC curve.

Operation	Is Sorting Necessary?	Why?
AUC Calculation	Yes (Mandatory)	Prevents chaotic, negative delta widths (Δ FPR) that corrupt the trapezoidal integration.
ROC Plotting	Yes (Mandatory)	Ensures the rendering engine draws a continuous, left-to-right line instead of a crisscrossed web of overlapping strokes.

```
In [20]: # 1. Obtain the indices that sorts the FPR tensor in ascending order
my @sorted_indices = sort { $fprs->[$a] <=> $fprs->[$b] } 0 .. $#fprs;

# 2. Sort both vectors using array slice over the @sorted_indices
my @sorted_fprs = @$fprs[@sorted_indices];
my @sorted_tprs = @$tprs[@sorted_indices];

# 3. Visual verification of the sort of FPR aligned with TPR
printf "sorted fprs:\n%s\n", dump @sorted_fprs[20 .. 30];
printf "sorted tprs:\n%s\n", dump @sorted_tprs[20 .. 30];
```

```
sorted fprs:
```

```
(  
  "0.00",  
  "0.00",  
  "0.00",  
  "0.00",  
  "0.00",  
  0.01,  
  0.02,  
  0.02,  
  0.05,  
  0.07,  
  0.08,  
)
```

```
sorted tprs:
```

```
(  
  "0.00",  
  "0.00",  
  "0.00",  
  "0.00",  
  "0.00",  
  0.13,  
  0.19,  
  0.18,  
  0.23,  
  0.29,  
  0.34,  
)
```

```
Out[20]: 1
```

```
In [21]: # Then, calculate the AUC using the trapezoid rule  
my $auc = sm1->trapz(\@sorted_fprs, \@sorted_tprs);  
printf "Area under the ROC curve (AUC): %0.2f\n", $auc;
```

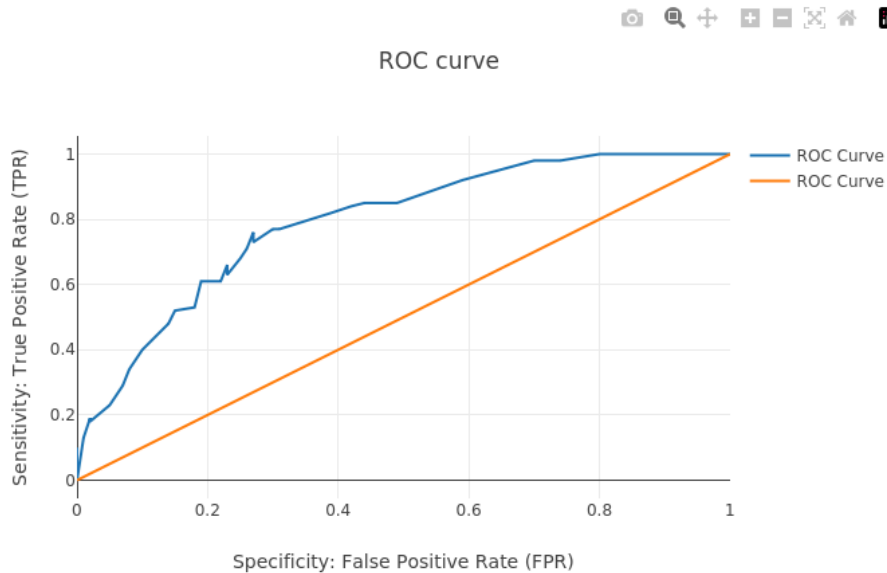
```
Area under the ROC curve (AUC): 0.79
```

```
Out[21]: 1
```

```
In [22]: # Plot the ROC curve using Chart::Plotly  
my $trace1 = new Chart::Plotly::Trace::Scatter(  
  x => \@sorted_fprs,  
  y => \@sorted_tprs,  
  mode => 'lines',  
  name => 'ROC Curve'  
);  
  
my $trace2 = new Chart::Plotly::Trace::Scatter(  
  x => [0, 1],  
  y => [0, 1],  
  mode => 'lines',  
  name => 'ROC Curve'  
);  
  
my $chart = new Chart::Plotly::Plot(  
  traces => [$trace1, $trace2],  
  layout => {  
    title => 'ROC curve',  
    xaxis => { title => 'Specificity: False Positive Rate (FPR)' },  
    yaxis => { title => 'Sensitivity: True Positive Rate (TPR)' }  
  }  
);
```

```
# Show the graph directly in IPerl
IPerl->display($chart);
```

```
In [23]: sml->embedplot($chart, width=>900, height=>450); # Este comando es de uso exclusivo de
```



4.3 Extensions

You have only seen a small sample of the most widely used performance metrics. There are many other performance metrics that you may require. Below is a list of 5 additional performance metrics that you may wish to implement to extend this tutorial

- Precision for classification.
- Recall for classification.
- F1 for classification.
- Area Under ROC Curve or AUC for classification.
- Goodness of Fit or R^2 (R squared) for regression.

4.4 Review

In this tutorial, you discovered how to implement algorithm prediction performance metrics from scratch in Perl. Specifically, you learned:

- How to implement and interpret classification accuracy.
- How to implement and interpret the confusion matrix for classification problems.
- How to implement and interpret mean absolute error for regression.
- How to implement and interpret root mean squared error for regression.